Runtime Assurance for Sampling-Based Optimizers in Flood Re-Planning: Exact Verification and Certified Escalation

Anonymous Author(s)
Affiliation withheld for double-blind review

***Abstract*—Quantum and annealing-based optimizers are proposed for disaster logistics, yet they return samples with no guarantee of validity, and the penalty encodings they need are often inexact. We study how a re-planning loop can use such samplers safely when evacuation groups must be reassigned to shelters as roads close and shelters fail. The loop verifies every sample exactly against the current state, falls back to a greedy rule, and, before handing a case to a human, asks an exact mixed-integer program (MILP) whether any valid plan exists. On 180 re-planning events of a flood digital twin (12 binary variables), dispatching the most frequent sample without checking would have sent an invalid plan in 29.1% of event draws for the Quantum Approximate Optimization Algorithm (QAOA) and 20.6% for simulated annealing; nearly all of these samples overfilled a shelter that the pairwise capacity penalty could not see, and the verified loop dispatched none. On 360 instances built on OpenStreetMap road networks of six Indian cities with up to 320 variables, the sampler's valid-plan rate on solvable instances fell from 67.9% at 8 groups to 0.0% at 32, and an uncertified loop escalated 11.9% of solvable cases to a human. The MILP settled 357 instances within 60 s. The other 3 timed out, yet all had valid plans, found within 118 s, so a solver time-out must never be read as infeasibility. An adaptive shot budget cut the shots per event from 64 to 18.4 with identical escalation decisions but fewer optimal QAOA plans.**

***Index Terms*—runtime assurance, quantum optimization, simulated annealing, mixed-integer programming, evacuation planning**

I. Introduction

When a flood closes roads and fills shelters, the assignment of evacuation groups to shelters has to be re-made, often many times in one night. The assignment is a small constrained optimization problem, and it is a common showcase for quantum and quantum-inspired optimizers, which take it as a quadratic unconstrained binary optimization (QUBO) problem [1]–[4]. Two properties of these tools matter more for an operations room than their speed. They return *samples*, so the same call can give a different, possibly invalid answer. And the QUBO they solve encodes capacity limits with penalties, which is exact only in special cases: a sample that satisfies every penalty can still overfill a shelter.

Safety-critical engineering has a standard answer to unreliable components: run them inside a *runtime-assurance* architecture, in which a simple verified controller checks or replaces the complex one [5], [6], as shields do for learned policies [7]. Three questions arise when the complex component is a sampling optimizer. How often would its output be unsafe without the check? When no sample is valid, how does the loop know whether to try harder or to call a human? And how many samples should it draw?

We answer them for flood re-planning. Our contributions are:

1) A runtime-assurance loop for sampling optimizers with an exact plan verifier, a greedy fallback and a *certified escalation* step that uses a mixed-integer program (MILP) to decide whether any valid plan exists (Fig. 1).
2) A measurement on twin episodes with QAOA [8], [9] and simulated annealing [10] of how often unverified dispatch would be unsafe, with ten independent sample draws per event, and a classification of every unsafe sample by the rule it breaks.
3) A city-scale study on OpenStreetMap road graphs of six Indian cities, beyond the reach of enumeration, showing where samplers and uncertified loops fail, what the certificate costs, and why a solver time-out is not a proof.
4) A shot-budget bound and an adaptive budget that cuts sampling without changing escalation decisions, with its measured cost in plan quality.

All QAOA results are classical simulations; we make no claim of quantum speed-up.

II. Related Work

Runtime assurance. The Simplex architecture switches from an unverified high-performance controller to a verified safety controller when the former leaves a safe envelope [5]; SOTER brings the pattern to robotics software [6], shields apply it to reinforcement learning [7], and runtime verification checks executions against a specification as they happen [11]. Levels-of-automation theory argues that automation should fall as the cost of an error rises [12]. Our verifier is the switching logic, and the MILP plays the role of the verified controller of last resort.

Samplers as tools. Variational quantum algorithms are hybrid loops in which a classical optimizer calls a quantum circuit [13]; QAOA with constraint-preserving mixers keeps one-hot structure but still penalizes capacity [9], [14]. Quantum annealing has been applied to disaster resource allocation [3], and reviews stress that practical advantage in optimization is unproven [4]. Agent frameworks let language models call tools [15], [16], and runtime enforcement constrains the actions such

TABLE I
POSITION OF THIS WORK

Work	Unreliable part	Check	Escalation
Simplex, SOTER [5], [6]	controller	envelope	safe controller
Shielding [7]	RL policy	automaton	blocked action
QUBO for disasters [3]	annealer	–	–
This work	sampler	exact verifier	MILP-certified

agents may take [17]; these works do not check the plans an optimizer returns.

Evacuation planning. Shelter allocation has long been solved with integer programming [18]; modern MILP solvers such as HiGHS [19] make exact solutions cheap at the sizes of a city district. Repairing a plan rather than re-planning from scratch reduces disruption [20].

Table I summarizes the gap. We found no application of runtime assurance to sampling optimizers, and the sampler studies cited here report solution quality per call; none measures what happens when a loop acts on the answer.

III. Problem and Loop

A. Assignment

Groups $g = 1, \dots, G$ of size n_g are assigned to shelters $s = 1, \dots, S$ with remaining seats R_s. With travel cost c_{gs} on the best open path and a set $\mathcal{F}$ of forbidden pairs (no open path, failed shelter, group larger than the seats left), a plan $\mathbf{x} \in \{0,1\}^{G \times S}$ is valid when

$$V(\mathbf{x}) = \Big[\textstyle\sum_s x_{gs} = 1 \ \forall g\Big] \wedge \Big[x_{gs} = 0 \ \forall (g,s) \in \mathcal{F}\Big] \wedge \Big[\textstyle\sum_g n_g x_{gs} \le R_s \ \forall s\Big], \quad (1)$$

and the planner minimizes $\sum_{g,s} c_{gs} x_{gs}$ over valid plans. Samplers solve the QUBO with a per-group shifted cost, a one-hot penalty and a pairwise penalty $P\, x_{gs} x_{hs}$ for every two groups that together exceed R_s [2]. The pairwise term misses overloads by three or more groups whose pairs all fit, so a sample can satisfy every penalty and still violate (1).

B. Runtime-Assurance Loop

Algorithm 1 gives the loop. It verifies every sample with (1) against the state at dispatch time, keeps the cheapest valid one, falls back to a greedy rule (largest group first to its cheapest shelter with seats left), and only then asks the MILP

$$\min_{\mathbf{x}} \sum_{g,s} c_{gs} x_{gs} \quad \text{s.t. (1)}, \ x_{gs} \in \{0,1\}. \quad (2)$$

If (2) is infeasible, escalation to a human is *certified*: no valid plan exists. If it is feasible, the loop dispatches its plan instead of escalating. If the solver stops at its time limit, the loop escalates but marks the case *uncertified*, so the officer knows that a plan may exist. A loop that escalates whenever the sampler and greedy rule fail, without asking the MILP, is the natural baseline; we call it the uncertified loop.

Algorithm 1 Runtime-assurance re-planning at one event

```
draw shots in batches of b until a sample passes V or the budget N is used
x ← cheapest sample with V(x); if none, x ← greedy
if V(x) then
    dispatch x
else
    solve (2) with a time limit
    if feasible, dispatch its plan; if infeasible, escalate with the certificate
    if timed out, escalate marked as uncertified
end if
```

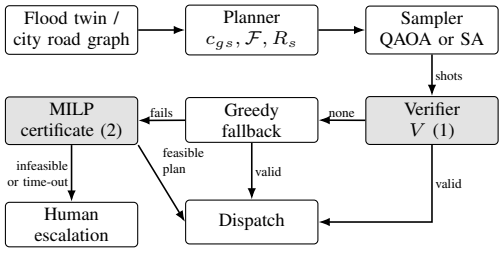


Fig. 1. Runtime-assurance loop. Shaded blocks are exact, and the sampler is never trusted. A human is called when the MILP proves that no valid plan exists, or, flagged as uncertified, when it runs out of time.

C. Shot Budget

If one shot is valid with probability q, the chance that N independent shots contain no valid plan is $(1-q)^N$, so

$$N \ \ge \ \frac{\ln \epsilon}{\ln(1-q)} \quad (3)$$

shots make that chance at most ϵ. Since q is unknown and changes with every event, the loop draws batches of $b = 8$ shots and stops at the first batch with a valid sample, up to $N = 64$. A wrong guess of q then costs shots or plan quality, while the verifier and certificate still stand behind every decision.

IV. Experimental Setup

Twin episodes. We replay 30 seeded episodes of a flood digital twin with four groups and three shelters (12 binary variables): the initial state, closure of road R5, a shelter surge of half its capacity, a further random closure, the failure of shelter A and, as a final stress, the failure of shelter B (180 re-planning events, 27 of them without any valid plan). QAOA uses a ring-XY mixer at depth 3, trained per event by COBYLA with layer-wise interpolation [21]; simulated annealing runs on the same QUBO. Each event is sampled ten times with independent seeds.

City instances. We built 360 instances on the OpenStreetMap road graphs of Delhi, Chennai, Mumbai, Kolkata, Guwahati and Patna, with the mapped zones and ten shelters per city, $G \in \{8, 16, 32\}$ groups (80–320 variables), demand of 60% or 90% of the seats left, and a disruption that closes the three most flood-exposed roads and fails one shelter. Travel times follow the prototype's router (flood- and traffic-weighted length at 25 km/h); capacities are building-type estimates

TABLE II
TWIN EPISODES: 180 EVENTS × 10 DRAWS, 12 VARIABLES

Sampler, budget	Unsafe if unverified	Valid sample	Optimal dispatched	Shots used	False escal.
QAOA, 64 shots	29.1	97.1	93.1	64.0	0.0
QAOA, adaptive	28.8	97.1	68.8	18.0	0.0
SA, 64 shots	20.6	95.4	95.4	64.0	0.0
SA, adaptive	21.0	95.4	95.4	18.7	0.0

All in % except shots. False escal.: uncertified loop; the certified loop had none.

TABLE III
ONE TWIN EPISODE WITH QAOA (SEED 178, 10 DRAWS PER EVENT)

Event	Plan exists	P(valid) of state	Top sample unsafe	Plan source (draws)	Max. gap %
E0 initial	yes	0.98	0/10	sampler 10	0.0
E1 R5 closed	yes	0.98	0/10	sampler 10	0.0
E2 surge	yes	0.02	10/10	sampler 7, greedy 3	1.0
E3 closure	yes	0.00	10/10	greedy 10	5.3
E4 A failed	yes	0.00	10/10	greedy 8, sampler 2	5.3
E5 B failed	no	0.00	10/10	escalate 10	–

from the city data, and group sizes are synthetic. Simulated annealing draws 64 parallel chains per call, ten times per instance. Enumeration of 10^G plans is impossible here, so the MILP is the only exact reference.

Metrics. Unsafe dispatch: the most frequent sample, dispatched unverified, violates (1). False escalation: a case escalated although (2) has a solution. Optimality gap: excess travel cost of the dispatched plan over the MILP optimum. Times are wall-clock on a laptop running three instances in parallel. The draws of one event share its instance, so they are not independent samples of the event population.

V. RESULTS

A. Twin Episodes

Table II gives the twin results over 180 events and ten draws each. Dispatching the most frequent sample unchecked would have been unsafe in 29.1% of event draws for QAOA and 20.6% for simulated annealing. These include the 27 infeasible events, where any answer a sampler gives is wrong. The verified loop dispatched no invalid plan, escalated every infeasible event and no feasible one, and dispatched an optimal plan in 93.1% (QAOA) and 95.4% (annealing) of feasible draws. At this size the uncertified loop also behaved well (0.0% and 0.0% false escalations), because the greedy rule found a plan whenever the sampler did not. The MILP took a median of 6.9 ms against 1.4 ms for full enumeration, while one QAOA training took 13.0 s of simulation. Resampling events gives 95% intervals of 22.7–35.6% for QAOA and 14.3–26.4% for annealing. On the 153 solvable events, the top QAOA sample was unsafe more often than the top annealing sample in 24 events and less often in 7 (Wilcoxon signed-rank $p = 2.4 \times 10^{-4}$), so the difference is consistent across events and not an effect of a few draws.

Table III follows one episode with QAOA through its six events (seed 178, the one in which the greedy rule did the most work). In the first two events the trained state put almost all of its probability on valid plans. After the surge it put only 0.02 there, and after the second closure 0.00, so the most frequent sample was invalid in every draw from then on. After the surge the verifier still picked out a rare valid sample in some draws; otherwise the greedy rule supplied the plan, and no dispatched plan cost more than 5.3% above the optimum. When shelter B failed, the MILP proved that no plan existed and the loop escalated with a certificate.

TABLE IV
TWIN RESULTS BY EVENT: UNSAFE IF DISPATCHED UNVERIFIED, OPTIMAL UNDER THE VERIFIED LOOP (30 EVENTS EACH; % OF DRAWS)

Event	No valid plan	Unsafe		Optimal	
		QAOA	SA	QAOA	SA
E0 initial	0/30	0.0	1.0	100.0	100.0
E1 R5 closed	0/30	0.0	1.0	100.0	100.0
E2 surge	0/30	23.7	7.7	85.0	93.3
E3 closure	0/30	28.0	10.3	90.3	93.3
E4 A failed	0/30	33.0	13.3	89.3	90.0
E5 B failed	27/30	90.0	90.0	100.0	100.0

Table IV splits the twin results by event. In the initial state and after the first road closure the most frequent sample was almost always valid. The risk appeared once capacity became tight: after the shelter surge, a second closure and the failure of shelter A, the unsafe rate of QAOA rose from 23.7% to 33.0%, and that of annealing from 7.7% to 13.3%. After shelter B also failed, 27 of 30 events had no valid plan at all. Counting solvable events only, unverified dispatch would still have been unsafe in 16.6% (QAOA) and 6.5% (annealing) of draws.

B. City Scale

Table V and Fig. 2 give the city results. As the problem grew from 8 to 32 groups, simulated annealing produced a valid plan in 67.9% and then 0.0% of calls on solvable instances; Section V-C shows that at this scale most samples were not even complete assignments. Dispatching the top sample unchecked would have been unsafe in most calls at every size. With 8 groups at 90% demand each group is about as large as the free seats of an average shelter, and no instance had a valid plan. The uncertified loop escalated 11.9% of solvable cases overall (31.5% at 90% demand, 0.0% at 60%), because the greedy rule also fails when capacity is tight. Where the greedy rule did find a plan, its median cost was 17.3% above the optimum. Within the 60 s limit the MILP settled 357 of the 360 instances, finding the optimal plan or proving that none exists, in at most 23.8 s. The other 3, all with 32 groups at 90% demand, hit the 60 s limit with neither a plan nor a proof, so the loop would have escalated them without a certificate. Given up to 1800 s, the solver found an optimal plan for all 3 in at most 118 s, one of them in 1 s, so solve times vary with machine load. Treating a time-out as proof of infeasibility would have sent solvable cases to a human.

TABLE V
CITY INSTANCES ON OPENSTREETMAP ROAD GRAPHS (6 CITIES)

G	vars	demand	feasible inst.	valid sample %	unsafe %	false esc. uncert. %	MILP s (max)
8	80	60%	53/60	67.9	70.2	0.0	0.18
8	80	90%	0/60	–	100.0	–	0.04
16	160	60%	51/60	6.1	100.0	0.0	1.61
16	160	90%	45/60	1.3	100.0	52.4	16.03
32	320	60%	50/60	0.0	100.0	0.0	7.82
32	320	90%	49/60	0.0	100.0	12.2	23.79†

valid: verified plan in 64 SA shots; unsafe: top sample unchecked.
–: none solvable. †Solved runs; 3 hit the 60 s limit.

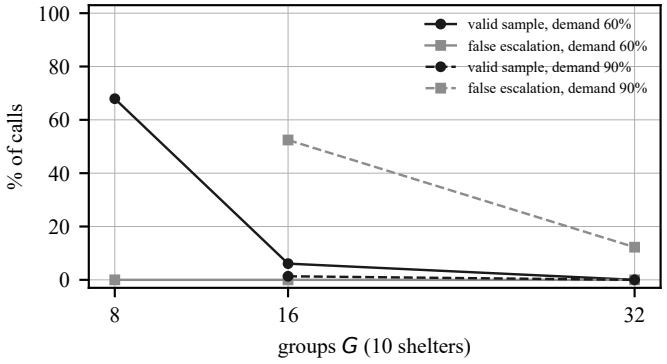


Fig. 2. City scale: share of calls in which the sampler produced a valid plan, and share of solvable cases an uncertified loop would escalate, against the number of groups.

Fig. 3 shows what the certificate costs. The median MILP time rose from 0.02 s at 80 variables to 0.74 s at 320, so at district scale a typical certificate takes under a second; the cost lies in the rare slow instances. Resampling the solvable instances puts the false-escalation rate of the uncertified loop at 8.1–15.9% (95% interval).

Table VI gives the same results per city. Unverified dispatch was unsafe in at least 87.7% of calls in every city. In Kolkata the three closures cut one of the six zones off from every shelter, so any instance with a group there has no valid plan, and only 4 of 60 were solvable. Delhi had the most false escalations (27.1%) and the weakest greedy rule, which found a plan in 72.9% of its solvable instances. The failures of the uncertified loop therefore depend on the city network as well as on the problem size.

C. Why Unverified Samples Fail

Table VII classifies every unsafe top sample by the first rule of (1) it breaks. On the twin,97.7% of the unsafe QAOA samples and 94.6% of the annealing samples overfilled a shelter although every pair of groups in it fitted, so the QUBO assigned them no penalty at all. Overloads with a penalized pair almost never reached the top, and the XY mixer never broke the one-hot rule. At city scale the picture changes: the annealer, run for a fixed 40 sweeps, left groups with no shelter or with several in 65.5% of unsafe samples at 8 groups, 94.2% at 16 and all of them at 32, while penalty-

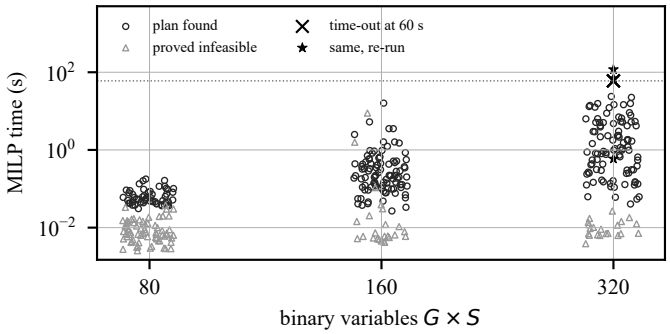


Fig. 3. Cost of the certificate: MILP solve time of every city instance (log scales, sizes jittered). Crosses mark the runs that hit the 60 s limit and stars the same instances re-run with a longer limit.

TABLE VI
CITY INSTANCES BY CITY (60 INSTANCES EACH; % UNLESS NOTED)

City	Solvable	Valid sample	Unsafe	False esc. uncert.	Greedy found	MILP s (median)
Mumbai	50/60	19.2	93.8	7.8	92.0	0.15
Chennai	50/60	20.6	87.7	11.4	88.0	0.19
Patna	50/60	7.0	96.7	4.0	96.0	0.11
Delhi	48/60	12.7	98.3	27.1	72.9	0.46
Guwahati	46/60	19.6	93.7	10.9	89.1	0.12
Kolkata	4/60	30.0	100.0	0.0	100.0	0.01

free overloads fell from 31.0% to none. A small instance fails through the encoding and a large one through the search, and the verifier catches both. The re-run that produced these classes reproduced the original flags in every twin draw and in 358 of 360 city instances (99.9% of draws); we attribute the few differences to floating-point ordering in the vectorized annealer.

D. Shot Budget

With the adaptive budget the twin loop used 18.4 shots per event on average instead of 64 and took the same escalation decision in every draw. Plan quality did change for QAOA: it dispatched an optimal plan in 68.8% of feasible draws against 93.1% with the full budget, because the loop keeps the cheapest valid sample of the first successful batch rather than of all 64 shots. Annealing dispatched an optimal plan whenever it found a valid one, with either budget, so its rate stayed at 95.4%. At city scale, where valid samples are rare, the adaptive rule used 58.2 shots per call and saved little. By (3), a sampler with $q = 0.05$ needs 90 shots for $\epsilon = 0.01$, more than the 64 used here.

VI. DISCUSSION AND LIMITATIONS

The study separates two questions that are often merged: whether a solver gives good answers, and whether a system built on it acts safely. The second holds by construction whenever every sample is verified, a classical plan is always available and the certificate finishes in time, and it holds for any sampler plugged in. The sampler decides how often the exact components have to step in. At city scale the pairwise

TABLE VII
FIRST RULE BROKEN BY THE UNSAFE TOP SAMPLES (% OF UNSAFE SAMPLES)

Sampler	Unsafe draws	One-hot broken	Forbidden pair	Overload penalized	Overload penalty-free
Twin, QAOA	524	0.0	1.9	0.4	97.7
Twin, SA	370	5.4	0.0	0.0	94.6
City, SA, $G = 8$	1020	65.5	3.4	0.1	31.0
City, SA, $G = 16$	1200	94.2	0.0	0.0	5.8
City, SA, $G = 32$	1200	100.0	0.0	0.0	0.0

capacity QUBO broke down and the sampler rarely contributed a valid plan, so an uncertified loop would have woken an officer for cases that a MILP solved in seconds.

The certificate has its own failure mode, the time limit. 3 city instances timed out at 60 s, and all of them had plans. An implementation that reads a time-out as "no plan" would have escalated them with a false certificate.

The results translate into four rules for anyone wiring a sampler into an operations loop:

1) Verify every sample against the state at dispatch time. Even on solvable twin events the top sample was unsafe in 6.5–16.6% of draws, and at city scale in most calls.
2) Ask an exact solver before calling a human. Without it, 11.9% of solvable city cases reached an officer.
3) Set the solver limit from the decision deadline and report a time-out as unknown, never as infeasible.
4) Keep the full shot budget when plan quality matters. The adaptive budget never changed an escalation, but it lowered QAOA's rate of optimal plans from 93.1% to 68.8%.

Threats to validity. Several design choices shape the numbers. The unsafe rate is measured on the most frequent sample, the answer a naive integration would act on; a loop that took the lowest-energy sample would see a different rate, but it would still need the verifier, because an overload that the pairwise term misses carries no penalty and can have lower energy than any valid plan. The penalty weight P is set once per instance from the cost range, as in the usual QUBO recipe [2], and was not tuned; a larger P changes how often samples break the penalized rules, while the overloads that the pairwise term misses stay invisible at any weight. QAOA was trained per event with a fixed budget and simulated without noise at 12 variables, and the city study uses simulated annealing as its only sampler, so we expect a noisy hardware sampler to produce valid samples less often. The draws of one event share its instance, which is why the intervals above resample events and instances rather than draws.

The external limits are those of the data. City demand is synthetic, capacities are estimates by building type, and every city instance uses the same kind of disruption: three closed roads and one failed shelter. The MILP is exact, but the slowest 320-variable instance needed 118 s, and regional problems will hit any fixed limit more often. Escalation was tested for infeasibility caused by failures and closures, and no officer took part in the evaluation, so the cost of a false escalation to a real operations room is not measured here.

VII. CONCLUSION

Sampling optimizers can join a flood re-planning loop safely if the loop never trusts them. An exact verifier checks every sample, a greedy rule and a MILP stand behind the sampler, and a human is called when the MILP proves that no plan exists or runs out of time, the second case flagged as uncertified. Without verification, the most frequent sample would have been unsafe in 29.1% of twin event draws for QAOA and 20.6% for annealing. On small instances these samples failed through the encoding, by overloads the pairwise penalty cannot see; on large ones they failed through the search, as incomplete assignments. Without certification, the loop escalated 11.9% of solvable city cases. With both, no invalid plan was dispatched, and the MILP settled 357 of 360 city instances within 23.8 s. We claim no quantum advantage. The results describe the checks that any such optimizer needs around it before it can act on a flood.

ACKNOWLEDGMENT

The authors thank the OpenStreetMap contributors for openly available map data. In line with the IEEE policy on AI-generated content, the authors disclose that Claude (Anthropic) was used to help edit and restructure the text of all sections and to draft the simulation, experiment and plotting scripts. All experiments use the authors' own software and fixed seeds. The authors checked every number, claim and reference and take full responsibility for the content.

REFERENCES

[1] A. Lucas, "Ising formulations of many NP problems," *Front. Phys.*, vol. 2, Art. no. 5, 2014, doi: 10.3389/fphy.2014.00005.
[2] F. Glover, G. Kochenberger, and Y. Du, "Quantum bridge analytics I: A tutorial on formulating and using QUBO models," *4OR*, vol. 17, no. 4, pp. 335–371, 2019, doi: 10.1007/s10288-019-00424-y.
[3] A. R. Cadeliña *et al.*, "D-Wave implementation of quantum annealing for optimal resource allocation in disaster response operation of Marikina City," *ECTI Trans. Comput. Inf. Technol.*, vol. 18, no. 1, pp. 24–33, 2024, doi: 10.37936/ecti-cit.2024181.253577.
[4] A. Abbas *et al.*, "Challenges and opportunities in quantum optimization," *Nat. Rev. Phys.*, vol. 6, pp. 718–735, 2024, doi: 10.1038/s42254-024-00770-9.
[5] L. Sha, "Using simplicity to control complexity," *IEEE Softw.*, vol. 18, no. 4, pp. 20–28, 2001, doi: 10.1109/MS.2001.936213.
[6] A. Desai, S. Ghosh, S. A. Seshia, N. Shankar, and A. Tiwari, "SOTER: A runtime assurance framework for programming safe robotics systems," in *Proc. 49th Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw. (DSN)*, 2019, pp. 138–150, doi: 10.1109/DSN.2019.00027.
[7] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu, "Safe reinforcement learning via shielding," in *Proc. AAAI Conf. Artif. Intell.*, vol. 32, no. 1, 2018, pp. 2669–2678, doi: 10.1609/aaai.v32i1.11797.
[8] E. Farhi, J. Goldstone, and S. Gutmann, "A quantum approximate optimization algorithm," arXiv:1411.4028, 2014.
[9] S. Hadfield, Z. Wang, B. O'Gorman, E. G. Rieffel, D. Venturelli, and R. Biswas, "From the quantum approximate optimization algorithm to a quantum alternating operator ansatz," *Algorithms*, vol. 12, no. 2, Art. no. 34, 2019, doi: 10.3390/a12020034.
[10] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by simulated annealing," *Science*, vol. 220, no. 4598, pp. 671–680, 1983, doi: 10.1126/science.220.4598.671.

[11] M. Leucker and C. Schallhart, "A brief account of runtime verification," *J. Log. Algebr. Program.*, vol. 78, no. 5, pp. 293–303, 2009, doi: 10.1016/j.jlap.2008.08.004.

[12] R. Parasuraman, T. B. Sheridan, and C. D. Wickens, "A model for types and levels of human interaction with automation," *IEEE Trans. Syst., Man, Cybern. A, Syst. Humans*, vol. 30, no. 3, pp. 286–297, 2000, doi: 10.1109/3468.844354.

[13] J. R. McClean, J. Romero, R. Babbush, and A. Aspuru-Guzik, "The theory of variational hybrid quantum-classical algorithms," *New J. Phys.*, vol. 18, no. 2, Art. no. 023023, 2016, doi: 10.1088/1367-2630/18/2/023023.

[14] Z. Wang, N. C. Rubin, J. M. Dominy, and E. G. Rieffel, "XY mixers: Analytical and numerical results for the quantum alternating operator ansatz," *Phys. Rev. A*, vol. 101, no. 1, Art. no. 012320, 2020, doi: 10.1103/PhysRevA.101.012320.

[15] S. Yao *et al.*, "ReAct: Synergizing reasoning and acting in language models," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023.

[16] T. Schick *et al.*, "Toolformer: Language models can teach themselves to use tools," in *Adv. Neural Inf. Process. Syst. 36*, 2023.

[17] H. Wang, C. M. Poskitt, and J. Sun, "AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents," in *Proc. IEEE/ACM 48th Int. Conf. Softw. Eng. (ICSE)*, 2026, pp. 2938–2950, doi: 10.1145/3744916.3764546.

[18] H. D. Sherali, T. B. Carter, and A. G. Hobeika, "A location-allocation model and algorithm for evacuation planning under hurricane/flood conditions," *Transp. Res. B, Methodol.*, vol. 25, no. 6, pp. 439–452, 1991, doi: 10.1016/0191-2615(91)90037-J.

[19] Q. Huangfu and J. A. J. Hall, "Parallelizing the dual revised simplex method," *Math. Program. Comput.*, vol. 10, no. 1, pp. 119–142, 2018, doi: 10.1007/s12532-017-0130-5.

[20] M. Fox, A. Gerevini, D. Long, and I. Serina, "Plan stability: Replanning versus plan repair," in *Proc. 16th Int. Conf. Automated Planning Scheduling (ICAPS)*, 2006, pp. 212–221.

[21] L. Zhou, S.-T. Wang, S. Choi, H. Pichler, and M. D. Lukin, "Quantum approximate optimization algorithm: Performance, mechanism, and implementation on near-term devices," *Phys. Rev. X*, vol. 10, no. 2, Art. no. 021067, 2020, doi: 10.1103/PhysRevX.10.021067.